BÀI TẬP TÌM KIẾM TRONG AI

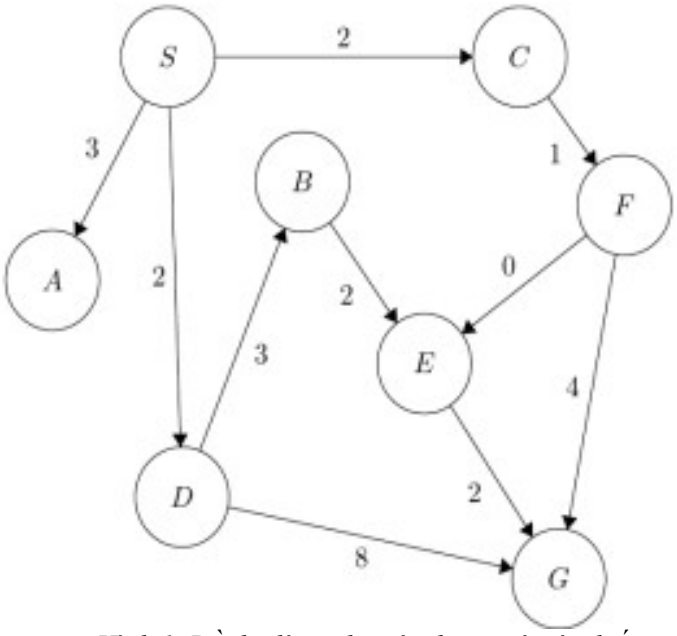
Họ và tên	
MSSV / Lớp	

Mục tiêu học tập

- Mô phỏng các thuật toán BFS, DFS và Iterative Deepening.
- Biểu diễn bài toán Tháp Hà Nội dưới dạng bài toán tìm kiếm.
- Thiết kế và phân tích heuristic cho A*.
- So sánh DFS, BFS, UCS, Greedy Search và A* trên cùng một bài toán.
- Phân tích các phát biểu lý thuyết về heuristic, A*, Minimax và Expectimax.

Bài 1. Các thuật toán tìm kiếm

Cho đồ thị có trạng thái bắt đầu S và trạng thái đích G như hình dưới đây.



Hình 1. Đồ thị dùng cho các thuật toán tìm kiếm

Yêu cầu: Với mỗi thuật toán, hãy ghi thứ tự các nút được đưa vào explored set, đường đi được trả về và tổng chi phí của đường đi. Nếu có hòa, phá hòa theo thứ tự alphabet của trạng thái cuối cùng trong đường đi.

1.1. Breadth-First Search (BFS)

Nội dung	Kết quả
Explored Set	S, A, C, D
Path	S -> D -> G

Total Cost	10

Giải quyết vấn đề bằng tìm kiếm **2** / **8**

1.2. Depth-First Search (DFS)

Nội dung	Kết quả
Explored Set	S, A, C, F, E
Path	S -> C -> F -> E -> G
Total Cost	5

1.3. Iterative Deepening Search (IDS)

Giới hạn độ sâu	Các nút được duyệt
0	S
1	S, A, C, D
2	S, A, C, F, D, B, G
...	

Path cuối cùng: S -> D ->
G_____

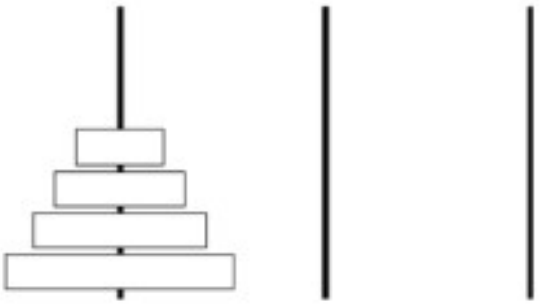
Total Cost: _____10_____

Câu hỏi thảo luận: BFS và DFS khác nhau ở cấu trúc frontier và tính tối ưu như thế nào?

BFS dùng hàng đợi kiểu FIFO nên luôn ưu tiên mở rộng các nút ở tầng nông trước, nhờ đó đảm bảo tìm được đường đi ngắn nhất tính theo số bước hoặc chi phí nếu các cạnh bằng nhau. Ngược lại, DFS sử dụng cấu trúc LIFO hay hàng đợi dạng Stack nên sẽ đi sâu theo một nhánh cho đến tận cùng mới quay lại, vì thế không đảm bảo tính tối ưu.

Bài 2. Tháp Hà Nội - Tower of Hanoi

Bài toán gồm n đĩa có kích thước khác nhau và ba cọc. Ban đầu các đĩa được xếp trên một cọc theo thứ tự từ lớn đến nhỏ. Mỗi lần chỉ được di chuyển một đĩa và không được đặt đĩa lớn lên trên đĩa nhỏ. Mục tiêu là chuyển toàn bộ đĩa sang cọc thứ ba.



Hình 2. Trạng thái ban đầu của bài toán Tháp Hà Nội

2.1. Biểu diễn trạng thái

Hãy đề xuất cách biểu diễn một trạng thái của bài toán. Biểu diễn phải cho biết mỗi đĩa đang ở cọc nào và đủ để xác định các hành động hợp lệ.

___Em đề xuất biểu diễn trạng thái bằng một bộ n phần tử dạng S = (d1, d2, ..., dn), trong đó di thuộc {1, 2, 3} đại diện cho cọc mà đĩa thứ i đang nằm (với d1 là đĩa nhỏ nhất và dn là đĩa lớn nhất). Cách biểu diễn này rất gọn và đủ để biết vị trí của từng đĩa.

_____

_____

_____

2.2. Kích thước không gian trạng thái

Với n đĩa, hãy xác định số trạng thái có thể có theo cách biểu diễn của bạn và giải thích cách suy luận.

- Với n đĩa, mỗi đĩa có thể đặt độc lập ở 1 trong 3 cọc. Do trên mỗi cọc các đĩa bắt buộc phải xếp theo thứ tự đĩa nhỏ nằm trên đĩa lớn nên không có sự hoán đổi thứ tự giữa các đĩa trên cùng cọc. Vậy số trạng thái có thể có là 3^n.

2.3. Trạng thái bắt đầu

Hãy viết trạng thái khởi đầu S0 theo cách biểu diễn đã chọn.

__Trạng thái ban đầu S0 = (1, 1, ..., 1), nghĩa là tất cả n đĩa đều đang nằm ở cọc 1.

2.4. Các hành động hợp lệ

Từ một trạng thái bất kỳ, hãy mô tả tập Actions(s). Một hành động hợp lệ phải thỏa mãn các điều kiện nào?

_Hành động chuyển đĩa i từ cọc c_from sang cọc c_to là hợp lệ khi thỏa hai điều kiện: đĩa i là đĩa nằm trên cùng ở cọc c_from, và cọc c_to hiện tại trống hoặc có đĩa trên cùng lớn hơn đĩa i.

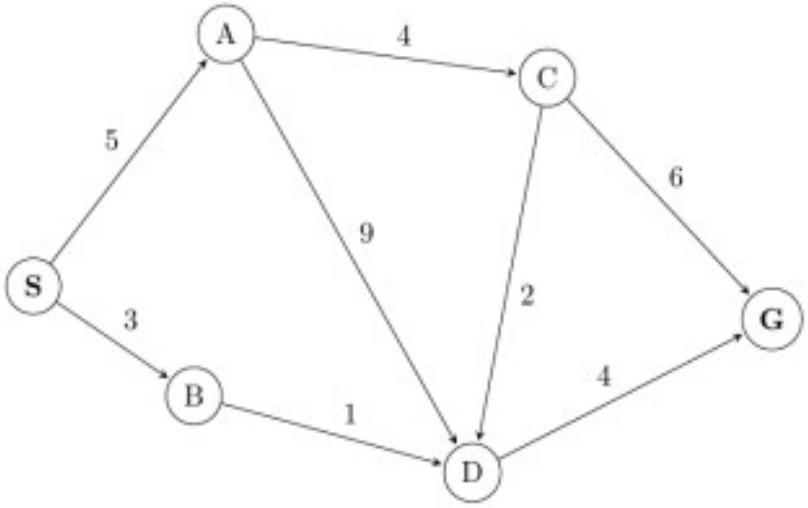
2.5. Goal Test

Hãy mô tả hàm goalTest(state): khi nào hàm trả về True?

Hàm goalTest(state) trả về True khi S = (3, 3, ..., 3), tức là toàn bộ n đĩa đã được chuyển sang cọc 3 thành công.

Bài 3. Thiết kế và phân tích Heuristic

Xét đồ thị dưới đây. Hãy thiết kế heuristic và phân tích ảnh hưởng của heuristic đến thuật toán A*.



Hình 3. Đồ thị dùng để thiết kế heuristic

Công thức A*: f(n) = g(n) + h(n), trong đó g(n) là chi phí thực tế từ S đến n và h(n) là chi phí ước lượng từ n đến đích.

3.1. Thiết kế heuristic admissible và consistent

Điền giá trị h(n) cho từng nút sao cho heuristic phù hợp với yêu cầu của nhóm.

3

Giải quyết vấn đề bằng tìm kiếm **4** / **8**

Node	h(n)
S	8
A	10
B	5
C	6
D	4
G	0

3.2. Chạy A* với heuristic đã thiết kế

Explored Set: S, B, D

_____

_____

Path:S -> B -> D -> G

Total Cost:8

3.3. Heuristic admissible nhưng không consistent

Đề xuất một bộ giá trị h(n) admissible nhưng không consistent. Chỉ ra ít nhất một cạnh vi phạm điều kiện h(n) <= c(n,n') + h(n').

_Ví dụ chọn h(S) = 8, h(A) = 0, h(B) = 0, h(C) = 6, h(D) = 0, h(G) = 0. Bộ giá trị này không vượt quá chi phí thực tế nên vẫn admissible. Tuy nhiên trên cạnh C -> D có chi phí c(C, D) = 2, ta thấy h(C) = 6 lớn hơn c(C, D) + h(D) = 2 + 0 = 2, do đó vi phạm điều kiện consistent._____

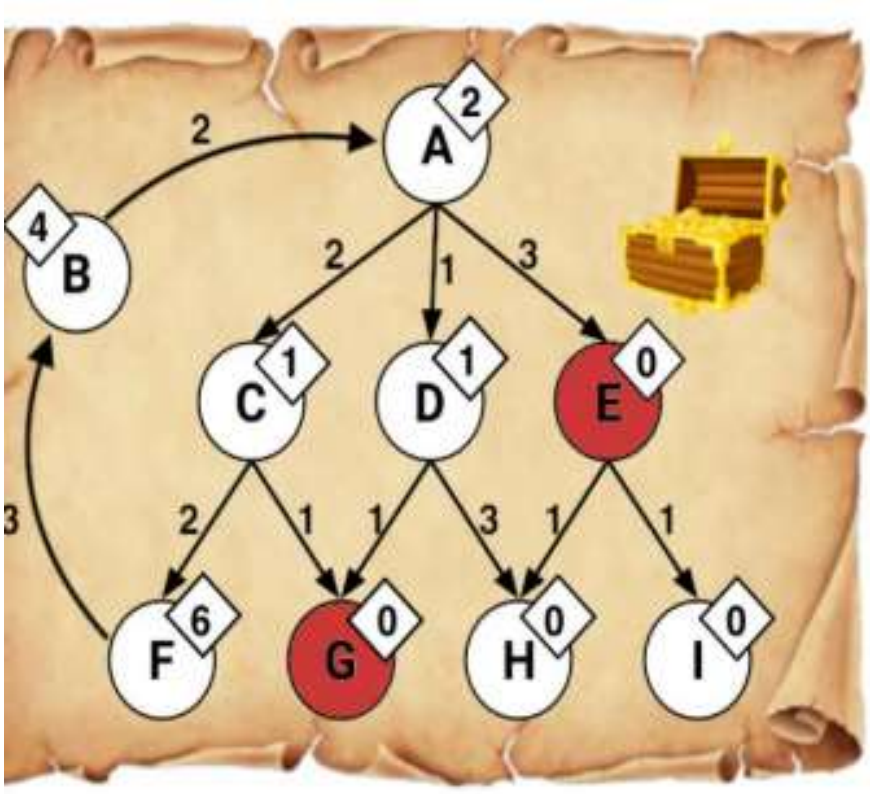
3.4. Câu hỏi mở rộng

Giải thích vì sao một heuristic consistent thì cũng admissible, giả sử h(G) = 0.

_____Khi một heuristic thỏa mãn tính consistent thì với mọi bước đi từ nút n đến nút n' ta luôn có h(n) <= c(n, n') + h(n'). Khi cộng dồn bất đẳng thức này dọc theo đường đi ngắn nhất từ n đến đích G và lưu ý h(G) = 0, ta thu được h(n) luôn nhỏ hơn hoặc bằng tổng chi phí thực tế h*(n). Vì vậy heuristic chắc chắn cũng admissible

Bài 4. Treasure Hunting (Tìm kho báu)

A là đảo xuất phát. Các nút màu đỏ là các đảo có kho báu; chỉ cần tới một trong các đảo đó. Mũi tên biểu diễn hành động có hướng; số trên cạnh là action cost. Giá trị trong hình thoi là heuristic ước lượng chi phí tối thiểu từ nút hiện tại tới một kho báu.



Hình 4. Bản đồ tìm kho báu (giữ nguyên hình gốc)

Quy tắc phá hòa: khi lấy phần tử khỏi frontier, nếu hòa thì chọn theo thứ tự alphabet.

4.1. Depth-First Search

Explored Set = {A, C, F, B}, Path = A -> C -> G, Cost = 3

4.2. Breadth-First Search

Explored Set = {A}, Path = A -> E, Cost = 3

Giải quyết vấn đề bằng tìm kiếm **6** / **8**

4.3. Uniform-Cost Search

Explored Set = {A, D, C}, Path = A -> D -> G, Cost = 2

4.4. Greedy Search

Explored Set = {A}, Path = A -> E, Cost = 3

4.5. A* Search

Explored Set = {A, D}, Path = A -> D -> G, Cost = 2

Bảng tổng hợp so sánh

Thuật toán	Tiêu chí chọn node	Path	Cost
DFS	Độ sâu		3

Giải quyết vấn đề bằng tìm kiếm **7** / **8**

BFS	Độ sâu nhỏ nhất		3
UCS	g(n)		2
Greedy	h(n)		3
A*	g(n)+h(n)		3

Bài 5. Câu hỏi Đúng/Sai

Với mỗi phát biểu, hãy ghi Đúng hoặc Sai và giải thích ngắn gọn. Nếu sai, nên đưa ra phản ví dụ. **Câu 1.** Depth-first search luôn mở rộng ít nhất nhiều node bằng A* sử dụng một admissible heuristic. Đúng / Sai: __SAI_____

Giải thích:DFS có thể đi may mắn đúng nhánh chứa đích ngay lập tức nên xét ít nút hơn A*.

Câu 2. Giả sử một quân xe trong cờ vua có thể đi bất kỳ số ô theo hàng hoặc cột trong một nước đi và không thể nhảy qua quân khác. Manhattan distance là một admissible heuristic cho số nước đi nhỏ nhất từ ô A đến ô B. Đúng / Sai: _____SAI_____

Giải thích: Quân xe chỉ tốn tối đa 2 nước đi là đến bất kỳ ô nào, trong khi khoảng cách Manhattan có thể lớn hơn 2 rất nhiều nên bị đánh giá cao hơn chi phí thực.

Câu 3. Euclidean distance là một admissible heuristic cho bài toán tìm đường của Pacman.

Đúng / Sai: _____ĐÚNG_____

Giải thích: Khoảng cách đường thẳng Euclidean giữa hai ô luôn nhỏ hơn hoặc bằng khoảng cách di chuyển thực tế theo ô lưới có chướng ngại vật.

Câu 4. Tổng của nhiều admissible heuristics vẫn luôn là một admissible heuristic.

Đúng / Sai: SAI

Giải thích: Ví dụ hai heuristic đều bằng chi phí thực h*(n), khi cộng lại sẽ thành 2*h*(n) vượt quá chi phí thực nên không còn admissible.

Câu 5. Admissibility của một heuristic cho A* kéo theo consistency.

Đúng / Sai: SAI

Giải thích: Một heuristic admissible vẫn có thể vi phạm bất đẳng thức tam giác giữa các nút kế tiếp nhau.

Câu 6. A* với graph search luôn tối ưu nếu heuristic admissible.

Đúng / Sai: SAI
Giải thích: Đối với A* Graph Search, heuristic bắt buộc phải thỏa mãn điều kiện consistent thì mới bảo đảm tìm được đường đi tối ưu.

Tổng kết nhanh

Nhóm	Thuật toán	Hàm/tiêu chí chính
Uninformed Search	BFS / DFS / IDS	Không sử dụng heuristic
Cost-based Search	UCS	g(n)
Informed Search	Greedy	h(n)

Giải quyết vấn đề bằng tìm kiếm **8** / **8**

Informed Search	A*	g(n)+h(n)

Ghi nhớ: A* dùng f(n) = g(n) + h(n). --

-o0o---

Tìm kiếm và Heuristic trong AI 1 / 5

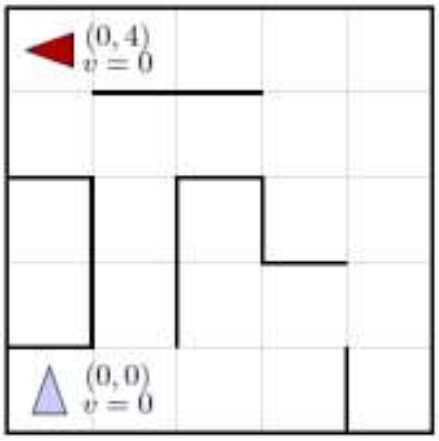
BÀI TẬP TÌM KIẾM VÀ HEURISTIC TRONG AI

Họ và tên	
MSSV / Lớp	

Mục tiêu học tập

- Mô hình hóa bài toán tìm kiếm trong AI.
- Vận dụng BFS, UCS, Greedy và A*.
- Phân tích và đánh giá heuristic.
- Phân biệt heuristic admissible và consistent.
- So sánh tính đầy đủ, tối ưu và hiệu quả của các thuật toán tìm kiếm.

Câu 1. Tìm kiếm và heuristic



Một tác tử dạng xe cần thoát khỏi mê cung như hình trên. Tác tử có hướng và tại mọi thời điểm luôn quay về một trong bốn hướng d ∈ {Bắc, Nam, Đông, Tây}. Trong mỗi bước, tác tử có thể tiến về phía trước với vận tốc có thể điều chỉnh hoặc thực hiện thao tác rẽ.

Các hành động di chuyển gồm tăng tốc, giữ tốc độ và giảm tốc. Sau hành động, tác tử di chuyển một số ô bằng vận tốc mới v'. Gọi v là vận tốc hiện tại và v' là vận tốc sau điều chỉnh:

• Tăng tốc (Faster): v' = v + 1

• Giảm tốc (Slower): v' = v - 1

• Giữ tốc độ (Maintain): v' = v

Các hành động rẽ gồm trái và phải, làm thay đổi hướng 90°. Chỉ được phép rẽ khi vận tốc bằng 0; sau khi rẽ vận tốc vẫn bằng 0.

• Rẽ trái: quay 90° ngược chiều kim đồng hồ.

• Rẽ phải: quay 90° theo chiều kim đồng hồ.

Ví dụ: nếu tác tử ở (0,0), quay về Bắc, v = 0 và cần đến (2,0), quay về Đông, v = 0 thì một chuỗi hành động hợp lệ là: rẽ phải, tăng tốc, giữ tốc độ, giảm tốc.

Các hành động không hợp lệ gồm:

• Hành động làm tác tử va vào tường.

• Làm v < 0 hoặc v > Vmax.

• Giữ tốc độ khi v = 0.

• Rẽ khi v ≠ 0.

Mục tiêu là tìm kế hoạch đưa xe dừng hẳn (v = 0) tại ô thoát, đúng hướng đích, với số hành động ít nhất. Chi phí đường đi được tính bằng số hành động đã thực hiện.

a) Giả sử tác tử đi theo nhánh ngoài cùng bên trái (đi qua ô (1,2)), từ trạng thái bắt đầu (0,0), hướng Bắc đến trạng thái đích (0,4), hướng Tây. Hãy viết chuỗi hành động ngắn nhất.

Tăng tốc -> Giảm tốc -> Rẽ phải -> Tăng tốc -> Giảm tốc -> Rẽ trái -> Tăng tốc -> Giữ tốc độ -> Giảm tốc -> Rẽ trái -> Tăng tốc -> Giảm tốc.

b) Nếu lưới có kích thước M × N và vận tốc tối đa là Vmax, kích thước không gian trạng thái là bao nhiêu? Giả sử mọi cấu hình đều có thể đạt được từ trạng thái bắt đầu.

M * N * 4 * (Vmax + 1) trạng thái. Vì xe có M * N vị trí, 4 hướng quay và Vmax + 1 giá trị vận tốc từ 0 đến Vmax.

c) Một "nút con" của trạng thái s là trạng thái s' có thể đạt được bằng một hành động hợp lệ từ s. Có thể tồn tại trạng thái không có nút con hay không? Nếu có, hãy cho ví dụ; nếu không, hãy giải thích.

☐ Có ☐ Không

Trả lời: Có.
Ví dụ xe đang di chuyển hướng thẳng vào bức tường ở ô cụt với vận tốc v > 0. Xe không thể tiến tiếp vì vướng tường, cũng không thể rẽ vì v phải bằng 0 mới rẽ được.

d) Hệ số phân nhánh lớn nhất của bài toán là bao nhiêu? Hãy cho một trạng thái ví dụ (x, y, hướng, vận tốc) đạt hệ số phân nhánh này và liệt kê các hành động có thể thực hiện.

Trả lời: b_max = 3.
Ví dụ trạng thái xe ở vị trí trống phía trước, đang chạy với vận tốc v = 1. Xe có 3 hành động hợp lệ là Tăng tốc, Giữ tốc độ và Giảm tốc.

e) Khoảng cách Manhattan từ vị trí hiện tại của tác tử đến vị trí cửa ra có phải là heuristic chấp nhận được không? Nếu không, hãy cho một trạng thái phản ví dụ, nêu giá trị heuristic và chi phí thực tế tối ưu đến đích.

☐ Có ☐ Không

Trả lời: Không.
Phản ví dụ: Xe ở (0,0) hướng Đông với v = 2, đích ở (2,0) v = 0. Khoảng cách Manhattan là 2, nhưng xe có thể đi đúng 2 ô chỉ bằng 1 hành động Giảm tốc (v' = 1). Vì h = 2 lớn hơn chi phí thực tế h* = 1 nên không admissible.

f) Heuristic h = khoảng cách Manhattan / Vmax có phải là admissible không? Giải thích. Nếu không, hãy cho phản ví dụ tương tự câu (e).

☐ Có ☐ Không

Trả lời: Có.
Vì chi phí di chuyển tối thiểu để vượt qua khoảng cách Manhattan ngay cả khi luôn chạy với vận tốc tối đa Vmax cũng không thể nhỏ hơn Manhattan / Vmax, do đó h(n) <= h*(n).

g) Nếu dùng heuristic không admissible trong A* Tree Search, tính đầy đủ của thuật toán có thể thay đổi không? Giả sử đồ thị hữu hạn và heuristic không âm.

☐ Có ☐ Không

Trả lời: Không. Thuật toán vẫn bảo đảm tính đầy đủ trên đồ thị hữu hạn.

h) Nếu dùng heuristic không admissible trong A* Tree Search, tính tối ưu của thuật toán có thể thay đổi không? Giả sử đồ thị hữu hạn và heuristic không âm.

☐ Có ☐ Không

Trả lời: Có. Thuật toán có thể trả về đường đi không tối ưu.

i) Những lý do nào sau đây có thể hợp lý khi dùng heuristic không admissible thay vì admissible? Chọn tất cả phương án phù hợp.

☐ Heuristic không admissible có thể dễ tính hơn, giúp giảm thời gian tính heuristic.

☐ Heuristic không admissible có thể gần chi phí thực hơn (dù đôi khi đánh giá cao), nhờ đó có thể mở ít nút hơn.

☐ Heuristic không admissible vẫn luôn tìm được đường đi tối ưu khi mọi chi phí thực không âm.

☐ Heuristic không admissible có thể được dùng để làm thuật toán hoàn toàn không xét một phần của đồ thị.

Câu 2. Các nút tìm kiếm

Xét tree search (không dùng tập explored) cho một bài toán tìm kiếm bất kỳ có hệ số phân nhánh tối đa b. Mỗi nút tìm kiếm n có chi phí tích lũy g(n), heuristic admissible h(n) và độ sâu d(n). Gọi n_c là nút đích có chi phí nhỏ nhất và n_s là nút đích nông nhất.

Với mỗi trường hợp, hãy viết một bất đẳng thức mô tả tập các nút được mở rộng trước khi thuật toán kết thúc. Không cần xử lý trường hợp hòa (tie). Nếu không có nút nào thỏa, ghi "không có".

a) Breadth-First Search (BFS).

Bất đẳng thức: $d(n) < d(n_s)$

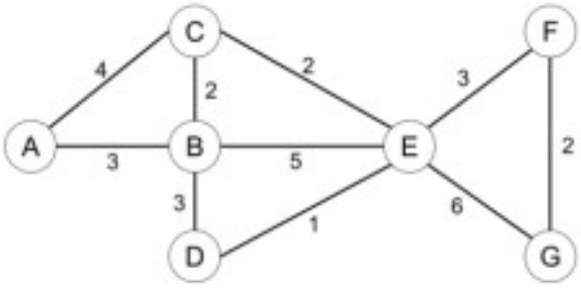
b) Uniform-Cost Search (UCS).

Bất đẳng thức: $g(n) < C^*$

c) A* Search, giả sử h là heuristic nhất quán.

Bất đẳng thức: $f(n) = g(n) + h(n) < C^*$

Câu 3. Tìm kiếm trên đồ thị



Node	h_1	h_2
A	12	11
B	6	7
C	9	6
D	3	4
E	3	5
F	2	1
G	0	0

Nút	h1	h2
A	12	11
B	6	7
C	9	6
D	3	4

E	3	5
F	2	1
G	0	0

Xét đồ thị vô hướng như hình. A là trạng thái bắt đầu và G là trạng thái đích. Chi phí của mỗi cạnh được ghi trên đồ thị. Bảng trên cung cấp hai heuristic h1 và h2. Khi có nhiều nút cùng ưu tiên, kết quả có thể phụ thuộc quy tắc phá hòa (tie-breaking).

a) Với mỗi chiến lược tìm kiếm đồ thị dưới đây, hãy đánh dấu X vào mọi đường đi mà thuật toán có thể trả về. Nếu kết quả không nằm trong các đường đi liệt kê, ghi đường đi đúng ở cột "Khác".

Thuật toán	A-C-E-G	A-C-E-F-G	A-B-D-E-F-G	Khác
UCS	☐	☐	☐	☐
Greedy với h1	☐	☐	☐	☐
Greedy với h2	☐	☐	☐	☐
A* với h1	☐	☐	☐	☐
A* với h2	☐	☐	☐	☐

b) Chi phí của đường đi tối ưu từ A đến G theo UCS là bao nhiêu?

Đáp án: 11

c) Heuristic h1 có admissible không? Có consistent không?

Admissible: ☐ Có ☐ Không Consistent: ☐ Có ☐ Không

d) Heuristic h2 có admissible không? Có consistent không?

Admissible: ☐ Có ☐ Không Consistent: ☐ Có ☐ Không

Câu 4. Trắc nghiệm và trả lời ngắn về tìm kiếm

a) Đánh dấu Đúng/Sai. Giả sử mọi hành động có chi phí ít nhất ε, với ε > 0, và mọi heuristic được dùng đều consistent.

Depth-First Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.

☐ Đúng ☐ Sai

Breadth-First Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.

☐ Đúng ☐ Sai

Iterative Deepening Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.

☐ Đúng ☐ Sai

Với mọi đồ thị không có chu trình, Graph Search luôn có frontier lớn hơn Tree Search.

☐ Đúng ☐ Sai

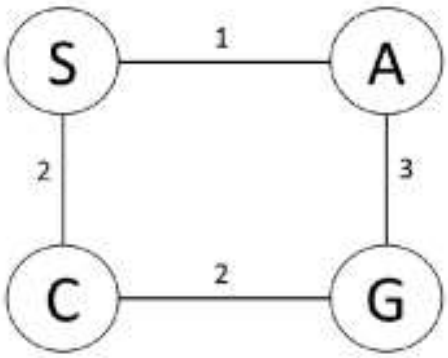
Iterative Deepening Graph Search có độ phức tạp thời gian của BFS và độ phức tạp bộ nhớ của DFS.

☐ Đúng ☐ Sai

Nếu h1(s) và h2(s) đều là heuristic consistent thì min(h1(s), h2(s)) cũng nhất thiết consistent.

☐ Đúng ☐ Sai

b) Xét đồ thị trạng thái dưới đây, S là bắt đầu và G là đích. Hãy điền các giá trị heuristic sao cho heuristic admissible nhưng không consistent.



Heuristic Function	
State	$h(s)$
S	
A	
C	
G	0

Trạng thái	h(s)
S	4
A	0
C	2
G	0

--- Hết ---